

CMSC3621 Lab Assignment 7 Report

1. Implementation

The hash table is implemented using a simple modulo based hash function:

$\text{index} = \text{data} \% \text{tableSize}$.

Collisions are handled using chaining. Each index in the hash table stores a deque, which allows multiple values to exist at the same index.

The `insert()` function computes the hash index and inserts the element at the front of the deque using `push_front()`, achieving $O(1)$ time complexity.

The `search()` function computes the index and traverses the deque at that index to check if the value exists. If found, it returns `true`; otherwise, `false`.

The `remove()` function also traverses the deque and removes the element using `erase()` if it is found.

2. Testing

The program was tested using the provided `main.cpp` file without any modifications. Multiple hash tables with different sizes (10, 7, 13, and 3) were created and tested.

Values were inserted into each hash table, and the structure was printed to verify that elements were placed in the correct indices. Search operations were performed for both existing and non-existing values, and removal operations were tested on the fourth hash table.

Screenshots:

```
// Hash Table ht1 (table size: 10)
| 0 | -> 10 -> 20
| 1 | ->
| 2 | -> 72 -> 92 -> 52
| 3 | ->
| 4 | ->
| 5 | ->
| 6 | -> 36
| 7 | ->
| 8 | -> 28 -> 48
| 9 | ->

Search 45 in ht1: 0
Search 92 in ht1: 1

// Hash Table ht2 (table size: 7)
| 0 | -> 28
| 1 | -> 92 -> 36
| 2 | -> 72
| 3 | -> 10 -> 52
| 4 | ->
| 5 | ->
| 6 | -> 48 -> 20

Search 12 in ht2: 0
Search 50 in ht2: 0

// Hash Table ht3 (table size: 13)
| 0 | -> 52
| 1 | -> 92
| 2 | -> 28
| 3 | ->
| 4 | ->
| 5 | ->
| 6 | ->
| 7 | -> 72 -> 20
| 8 | ->
| 9 | -> 48
| 10 | -> 10 -> 36
| 11 | ->
| 12 | ->

Search 36 in ht3: 1
Search 52 in ht3: 1

// Hash Table ht4 (table size: 3)
| 0 | -> 3 -> 102 -> 12
| 1 | -> 103 -> 13 -> 1
| 2 | -> 2 -> 101 -> 11

Search 101 in ht4: 1
Search 0 in ht4: 0 (REMOVED)
```

3. Correctness

The solution is correct because:

- The hash function correctly maps values to indices using modulo arithmetic.
- Collisions are handled properly using chaining with deque.
- The insert operation places elements in the correct bucket.
- The search function accurately determines whether a value exists.
- The remove function successfully deletes elements when present.
- The output matches the expected behavior shown in the assignment instructions.

4. Conclusion

This assignment demonstrates the use of hashing with chaining using STL deque. The implementation provides efficient insertion, search, and deletion operations. Through testing, the hash table was verified to behave correctly under different table sizes and input values.